

ota-telemetry-schema

Field	Value
Revision	2
Created	2026-06-07
Status	implemented
Part of	Helix OTA
Module path	github.com/HelixDevelopment/ota-telemetry-schema
Language	go (1.26)
License	Apache-2.0

Overview

ota-telemetry-schema (package `otatelemetry`) defines the telemetry event envelope, batch codecs, and the **pure health-signal derivations** the rollout engine consumes. It is schema + (de)serialization + pure derivation ONLY: it performs **no I/O** — no network, disk, HTTP, or database (`event.go`). Inputs arrive as `[]byte / io.Reader`; results leave as `[]byte / io.Writer`; everything else is pure functions over values.

It reuses the canonical wire contracts from [ota-protocol](#) — the six-value `TelemetryEvent` enum and the `TelemetryReport` shape — rather than re-declaring ids, so server and agents share one source of truth (declared as a `require` in `go.mod`).

Public API

Event envelope (`event.go`)

- `Event` — composes the canonical `otaprotocol.TelemetryReport (Report)` with transport-free annotations `Cohort`, `SystemHealth`, `ErrorMessage`.
 - `NewEvent(otaprotocol.TelemetryReport) Event` — construct from the wire report.
 - `Validate() error` — delegates to `otaprotocol.ValidateTelemetryReport`; on failure joins `ErrInvalidEvent` while preserving the underlying sentinel for errors.`Is`.
 - Accessors: `DeviceID()`, `DeploymentID()`, `EventType()`, `Timestamp()`.
 - Classification: `IsTerminal()`, `IsSuccess()`, `IsFailure()`.
- Re-exported event-id aliases (no second import needed):
`EventDownloadStarted`, `EventInstalling`, `EventInstalled`,
`EventVerifying`, `EventSuccess`, `EventFailure`.

- `AllEventTypes()` `[]otaprotocol.TelemetryEvent` — the canonical, lifecycle-ordered set.

Batch codecs (`codec.go`)

- `Batch` struct (`Events []Event`) — a set of events transmitted together (uncompressed JSON shape; compression is a transport concern handled elsewhere).
 - `Validate()` `error` — validates every event, annotating the first failure with its index.
- `EncodeBatch(Batch) ([]byte, error)` — validate-then-marshall; an invalid event cannot silently round-trip out (wraps `ErrEncode`).
- `DecodeBatch([]byte) (Batch, error)` — strict decode: rejects unknown fields and trailing data, then validates (wraps `ErrDecode` / `ErrInvalidEvent`).
- `DecodeBatchFrom(io.Reader) (Batch, error) / EncodeBatchTo(io.Writer, Batch) error` — stream variants; the caller owns the reader/writer lifecycle.

Health derivation (`health.go`)

- `Health` — the pure, derived health signal (totals, terminal count, successes/failures, `CountsByEvent`, `SuccessRate`, `FailureRate`, optional `DeploymentID/Cohort`).
- `DeriveHealth([]Event) Health` — aggregate counts and rates; rates use the count of **terminal** events as denominator; a mixed-scope set yields an unscoped (unlabeled) `Health`.
- `HealthThresholds` struct (`SuccessThreshold`, `ErrorThreshold`, fractions in `[0,1]`) with `Validate()` `error`.
- `(Health).Verdict(HealthThresholds) (Verdict, error)` — the rollout-gate decision enforcing the **safety invariant “halt wins over advance”**.
- `Verdict` enum: `VerdictHalt`, `VerdictAdvance`, `VerdictHold` (with `String()`).

Sentinel errors

`ErrInvalidEvent`, `ErrDecode`, `ErrEncode`, `ErrInvalidThresholds` (`event.go`).

Usage

```
package main
```

```
import (
    "fmt"
    "time"

    otaprotocol "github.com/HelixDevelopment/ota-protocol"
    otatelemetry "github.com/HelixDevelopment/ota-telemetry-
        schema"
)

func main() {
```

```

ev := otatelemetry.NewEvent(otaprotocol.TelemetryReport{
    DeviceID:      "dev-1",
    DeploymentID:  "dep-1",
    Event:         otatelemetry.EventSuccess,
    Progress:      100,
    Timestamp:     time.Now().UTC(),
})

batch := otatelemetry.Batch{Events: []otatelemetry.Event{ev}}
wire, err := otatelemetry.EncodeBatch(batch) // validates
first
if err != nil {
    panic(err)
}

got, err := otatelemetry.DecodeBatch(wire) // strict decode +
validate
if err != nil {
    panic(err)
}

health := otatelemetry.DeriveHealth(got.Events)
verdict, _ := health.Verdict(otatelemetry.HealthThresholds{
    SuccessThreshold: 0.95, ErrorThreshold: 0.05,
})
fmt.Println(health.SuccessRate, verdict) // 1 advance
}

```

Testing

```

cd submodules/ota-telemetry-schema
go vet ./...
go test ./...

```

The suite (`event_test.go`, `codec_test.go`, `health_test.go`) covers: event validation and accessor/terminal classification (`TestEventValidate`, `TestEventAccessors`, `TestEventTerminalClassification`); the canonical event set (`TestAllEventTypes`); batch encode/decode round-trips and **strict-decode rejection** of invalid events and malformed/trailing input (`TestEncodeDecodeBatchRoundTrip`, `TestEncodeBatchRejectsInvalidEvent`, `TestDecodeBatchFailures`, `TestEmptyBatchIsValid`); stream encode/decode error paths (`TestDecodeBatchFromReaderError`, `TestEncodeBatchToWriterError`); health derivation counts/rates incl. the no-terminal and mixed-scope cases (`TestDeriveHealthCountsAndRates`, `TestDeriveHealthNoTerminalEvents`, `TestDeriveHealthMixedScopeUnlabeled`); threshold validation, the halt-wins verdict logic, and the derive→verdict integration (`TestHealthThresholdsValidate`, `TestHealthVerdict`, `TestDeriveThenVerdictIntegration`).

Reusable building brick

This is a **reusable, independently versioned** Helix OTA building brick (HelixConstitution §11.4.28 — submodules-as-equal-codebase). Consume it via its module path `github.com/HelixDevelopment/ota-telemetry-schema`. It is project-not-aware and reusable by any fleet-telemetry consumer. Universal constitution rules are inherited via this repo's `CLAUDE.md` / `AGENTS.md` (`## INHERITED FROM Helix Constitution`).

Mirrors

- GitHub: <https://github.com/HelixDevelopment/ota-telemetry-schema>
- GitLab: <https://gitlab.com/helixdevelopment1/ota-telemetry-schema>